

Q1. Exception

Score: 0.00 TC: 0/3 passed

CODE

```
import java.util.Scanner;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        int[] arr= new int[5];
        for(int i=0;i<5;i++){
            if(sc.hasNextInt()){
                arr[i]=sc.nextInt();
            }
        }
        if(sc.hasNextInt()){
            int index=sc.nextInt();
            try{
                System.out.println("Element at index "+arr[index]);
            }catch(ArrayIndexOutOfBoundsException e){
                System.out.println("Exception: Invalid array index.");
            }
        }
        sc.close();
    }
}
```

INPUT

10 20 30 40 50
6

OUTPUT

Exception: Invalid array index.

Q2. IllegalArgumentException

Score: 0.00 TC: 0/3 passed

CODE

```
import java.util.Scanner;
public class Main{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        if(sc.hasNextInt()){
            int marks=sc.nextInt();
            try{
                if(marks<0||marks>100){
                    throw new IllegalArgumentException("Invalid marks.marks should be between 0 and 100.");
                }
                System.out.println("Valid marks: "+marks);
            }catch(IllegalArgumentException e){
                System.out.println("Exception: "+e.getMessage());
            }
        }
        sc.close();
    }
}
```

INPUT

120

OUTPUT

Exception: Invalid marks.marks should be between 0 and 100.

Q3. user-defined exception named WeakPasswordException.

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.Scanner;
public class Main{
    static class WeakPasswordException extends Exception{
        public WeakPasswordException(String message){
            super(message);
        }
    }
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        if(sc.hasNext()){
            String password=sc.next();
            try{
                if(password.length()<8){
                    throw new WeakPasswordException("Password must contain at least 8 characters.");
                }
                System.out.println("Password is Valid.");
            }catch(WeakPasswordException e){
                System.out.println("Exception: "+e.getMessage());
            }
        }
        sc.close();
    }
}
```

INPUT

java123

OUTPUT

Exception: Password must contain at least 8 characters.

Q4. Email Validation

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.Scanner;
public class Main{
    static class InvalidEmailException extends Exception{
        public InvalidEmailException(String message){
            super(message);
        }
    }
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        if(sc.hasNext()){
            String email=sc.next();
            try{
                if(!email.contains("@")){
                    throw new InvalidEmailException("Invalid email address.");
                }
                System.out.println("Valid email address.");
            }catch(InvalidEmailException e){
                System.out.println("Exception: "+e.getMessage());
            }
        }
        sc.close();
    }
}
```

INPUT

student.gmail.com

OUTPUT

Exception: Invalid email address.

Q5. Banking Transaction Using Exception Propagation

Score: 0.00 TC: 0/3 passed

CODE

```
import java.util.Scanner;
public class Main{
    static class InsufficientBalanceException extends Exception{
        public InsufficientBalanceException(String message){
            super(message);
        }
    }
    public static void processTransaction(double balance,double amount)throws InsufficientBalanceException{
        if(amount>balance){
            throw new InsufficientBalanceException("Withdrawal amount exceeds available balance.");
        }
    }
    public static void validateBalance(double balance,double amount) throws InsufficientBalanceException{
        processTransaction(balance,amount);
    }
    public static void withdraw(double balance,double amount) throws InsufficientBalanceException{
        validateBalance(balance,amount);
    }
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        if(sc.hasNextDouble()){
            double balance=sc.nextDouble();
            double amount=sc.nextDouble();
            try{
                withdraw(balance,amount);
                System.out.println("Transaction successful.Remaining balance: "+(balance-amount));
            }catch(InsufficientBalanceException e){
                System.out.println("Exception propagated to main().");
                System.out.println("InsufficientBalanceException: "+e.getMessage());
            }
        }
        sc.close();
    }
}
```

INPUT

5000
7000

OUTPUT

Exception propagated to main().
InsufficientBalanceException: Withdrawal amount exceeds available balance.